

**UNIVERSITY OF SÃO PAULO
SÃO CARLOS SCHOOL OF ENGINEERING**

**INSTRUCTION MANUAL FOR CONSTRUCTING URDF MODELS
PRACTICAL GUIDE FOR MODELING AND STRUCTURING IN ROS 2**

FELIPE PEREIRA FURLANETO

ffurlaneto@usp.br

Technical instruction manual focused on the development, parameterization, and validation of URDF files for the kinematic and dynamic simulation of robots.

**SÃO CARLOS
2026**

Contents

1	Preparation of Parts and Three-Dimensional Meshes	2
1.1	Structural Organization and Mesh Preparation in Blender	3
1.1.1	Step 1: Import and Environment Configuration	3
1.1.2	Step 2: The Scale Challenge	4
1.1.3	Step 3: Correcting Geometric Centers and Joint Articulation Points (Origin)	5
1.1.4	Step 4: Joining Static Parts (<i>Join</i>)	8
1.1.5	Step 5: Exporting to COLLADA (.dae)	10
1.1.6	Importing, Optimizing, and Customizing STL Files	12
2	Folder Creation, URDF Assembly, and Environment Configuration	16
2.0.1	Project Folder Structure	16
2.0.2	Configuration of Manifesto and Build Files	17
2.0.3	Understanding the Anatomy of URDF: What are Links and Joints? . . .	20
2.1	The Golden Rule of URDF Assembly (Relative Coordinate Reference)	24
3	Practical Code Example	27
3.0.1	Line-by-Line Analysis of the Example Code	28
4	Environment Automation with Launch Files	30
4.0.1	The Role of Core Initialization Nodes	30
4.0.2	Practical Example: Launch Script (<code>display.launch.py</code>)	30
4.0.3	Analysis of the Spawn Process	32
5	Adaptation for Isaac Sim	34
5.0.1	How to Convert Xacro Files to Static URDFs	34

1 PREPARATION OF PARTS AND THREE-DIMENSIONAL MESHES

The *Unified Robot Description Format* (URDF) consists of an XML specification for the native representation of robotic models in the ROS ecosystem. The scope of this manual covers the standardization and best practices in the development of these structures.

For structuring a custom assembly file, using the *Blender* computer modeling software is recommended, although certain technical caveats must be strictly observed to ensure the optimal functioning of your simulation file:

1. **Version Compatibility:** The use of the specific version 4.5.5 LTS is recommended, since variations in the software's development pipeline can alter the integrity of exporting Collada (.dae) files containing integrated material properties and color textures;
2. **Consistency of Measurement Units:** Due to the nature of *Blender* as an ecosystem oriented towards 3D modeling and Computer Generated Imagery (CGI), its factory default metric unit is the meter (with a scale factor equal to 1.0). Consequently, when importing meshes generated in millimeters from engineering CAD software — such as *Autodesk Inventor* or *Autodesk Fusion 360* —, a direct dimensional distortion occurs. For example, a component designed with a length of 23 mm will be erroneously interpreted in the *Blender* environment as having a length of 23 m. The procedures for correction will be detailed in the subsequent sections;
3. **Alignment of the Geometric Center (Origin):** During the process of importing external files, it is common for the origin point or geometric center of the object (*origin point*) to be shifted relative to the global axes. It is up to the developer to manually reposition and center this origin in the software, since the correct location of this frame is a mandatory requirement for determining the rotation axes of joints and the correct distribution of inertial properties in the final URDF file;
4. **Purpose of the Modeling Environment:** It is emphasized that *Blender*'s role in this pipeline is restricted to the spatial organization of components, scale verification, and geometric center adjustments. For the modeling and geometric design of complex mechanical parts, the dedicated use of industrial CAD software of the designer's choice is recommended;
5. **Intermediate Export Formats:** When exporting components from the engineering CAD software to *Blender*, two distinct approaches are suggested depending on the workflow: exporting in .stl format (if the customization of colors and materials is to be performed later within *Blender* itself) or in .obj format (when it is desirable to preserve and directly import the color definitions and appearances structured in the original modeling).

1.1 Structural Organization and Mesh Preparation in Blender

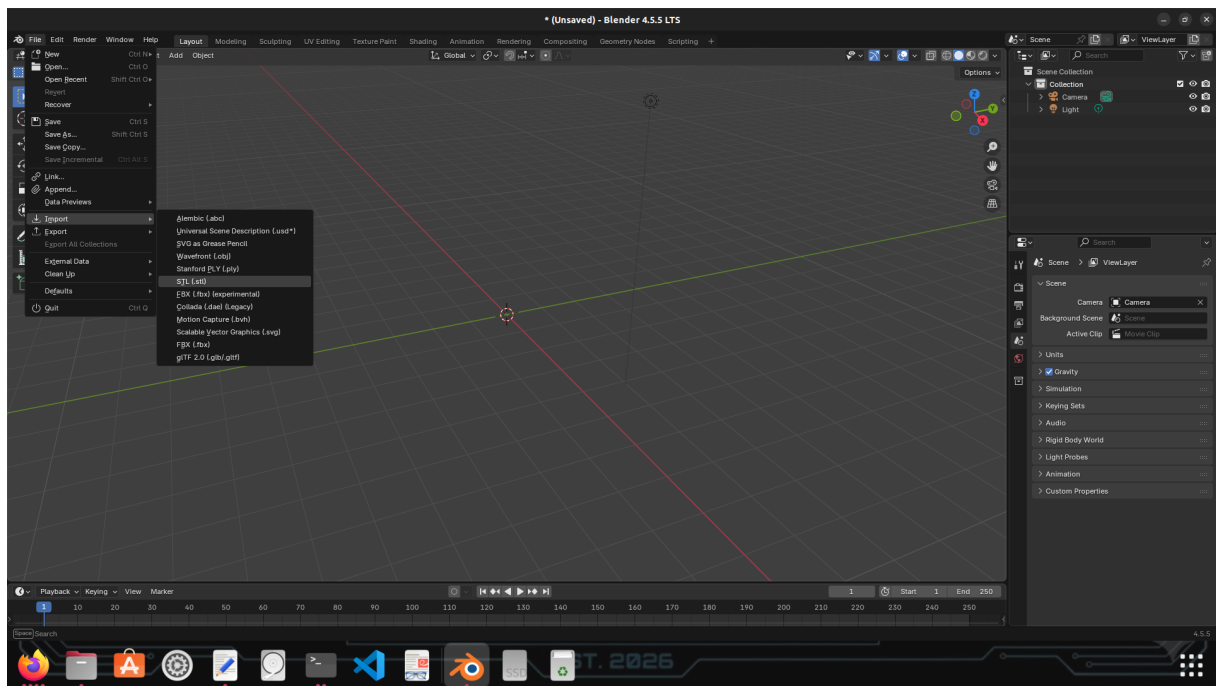
For a 3D model to be correctly imported into a URDF file, simply drawing or importing the parts is not enough. ROS 2 and simulators (such as Gazebo) demand technical rigor regarding the **scale**, the **orientation**, and, fundamentally, the **position of the origin (pivot point)** of each *link*.

Below, we describe the ideal process of importing, correcting scale, aligning geometric centers (origins), and exporting using Blender.

1.1.1 Step 1: Import and Environment Configuration

The first step consists of bringing the individual parts of each robot element into the Blender scene (as shown in **Figure 1.1**).

Figure 1.1: Import interface in Blender



When importing each part, two initial actions are fundamental to the workflow:

1. **Visualization:** Change the display mode to **Viewport Shading** (as shown in **Figure 1.2**) to better visualize the surfaces and textures of the model.
2. **Properties Panel:** Press the **N** key on the keyboard to enable the *Sidebar* (side panel). In the *Transform* tab, you will have immediate, real-time access to the **Location**, **Rotation**, **Scale**, and **Dimensions** of the selected object (as shown in **Figure 1.3**).

Figure 1.2: Changing the display mode

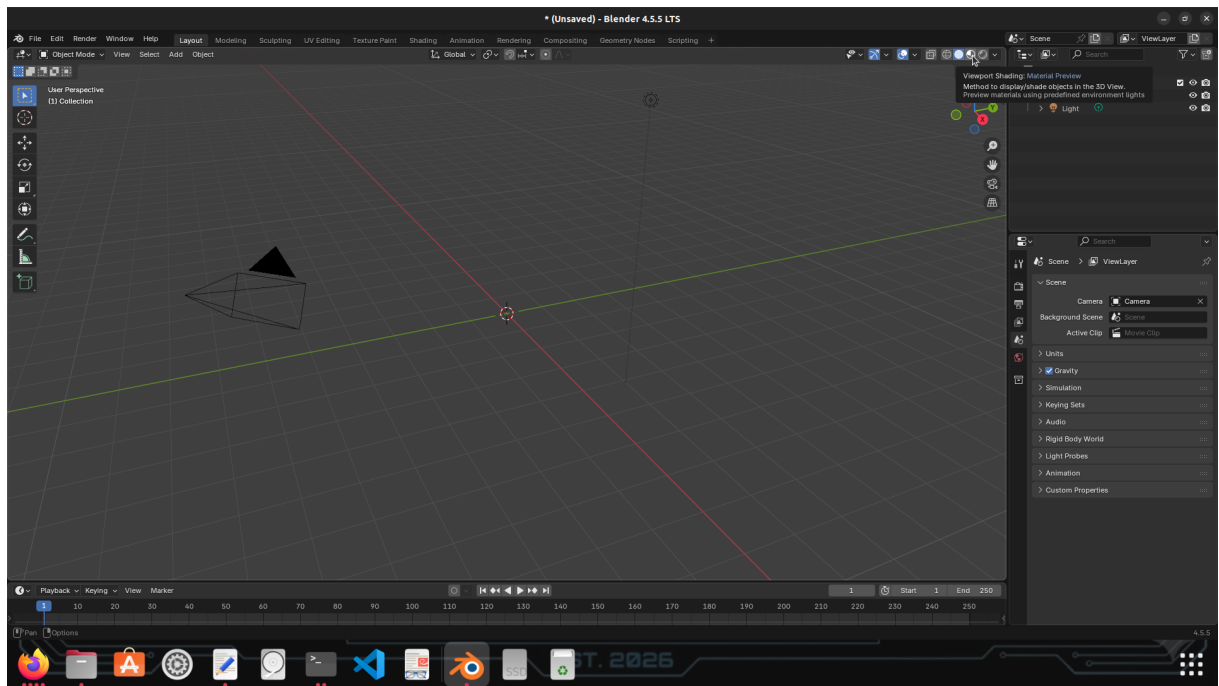
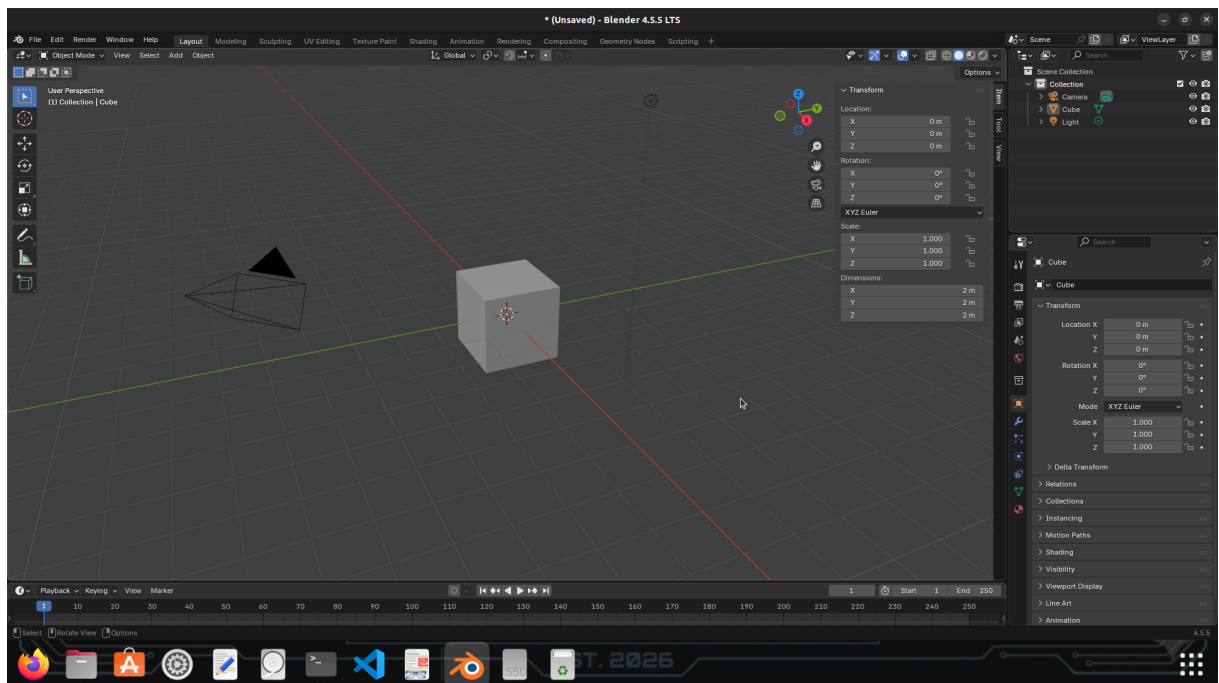


Figure 1.3: Sidebar enabled



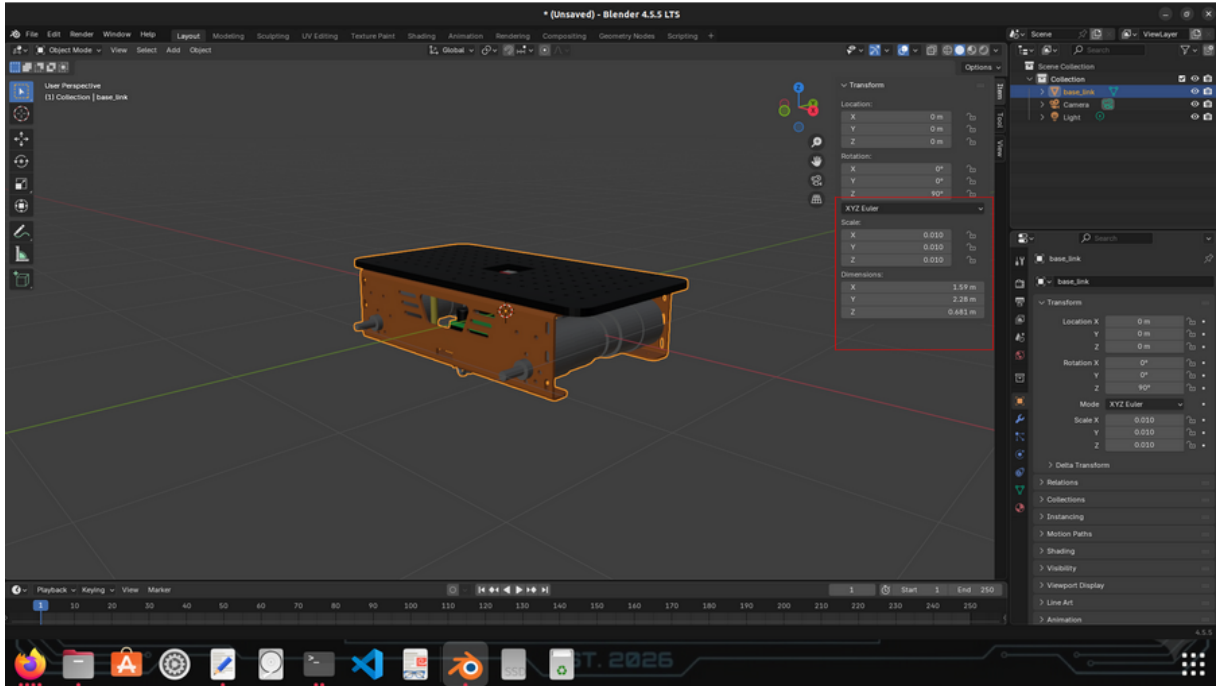
1.1.2 Step 2: The Scale Challenge

Blender and CAD software frequently present scale incompatibilities upon import (usually due to millimeter-to-meter conversion). To resolve this, check the *Dimensions* field. If the part is

outside its actual size, adopt one of the following recommended strategies:

- Change the **Scale** field value directly to 0.001 in Blender.
- **Alternative:** Apply a scale of 0.01 in Blender and, subsequently, in the URDF file, define the scale tag of the *visual/collision* elements as 0.1 (as shown in **Figure 1.4**).

Figure 1.4: Part with proper scale



1.1.3 Step 3: Correcting Geometric Centers and Joint Articulation Points (Origin)

By default, importing can shift the object's origin point (the orange dot highlighted in **Figure 1.5**). In the URDF, this orange dot will be the **origin of the link** and the exact position where the joint (*joint*) will connect. Therefore, it must be positioned with high precision.

Figure 1.5: Object origin point



For Fixed Parts

The orange dot must be located exactly at the geometric center of the part.

For Moving Parts (Robot Arm Links, Wheels, etc.)

The orange dot **must** be positioned exactly on the rotation or translation axis of the part. For example, as shown in **Figure 1.6**, if the joint rotates around the Y axis (green line), the orange dot should be positioned at the start of the link's elbow connection. The same applies to wheels (angular movement), where the point must sit at the exact center of the wheel axis (**Figure 1.7**).

Figure 1.6: Robotic arm joint with rotation axis on Y

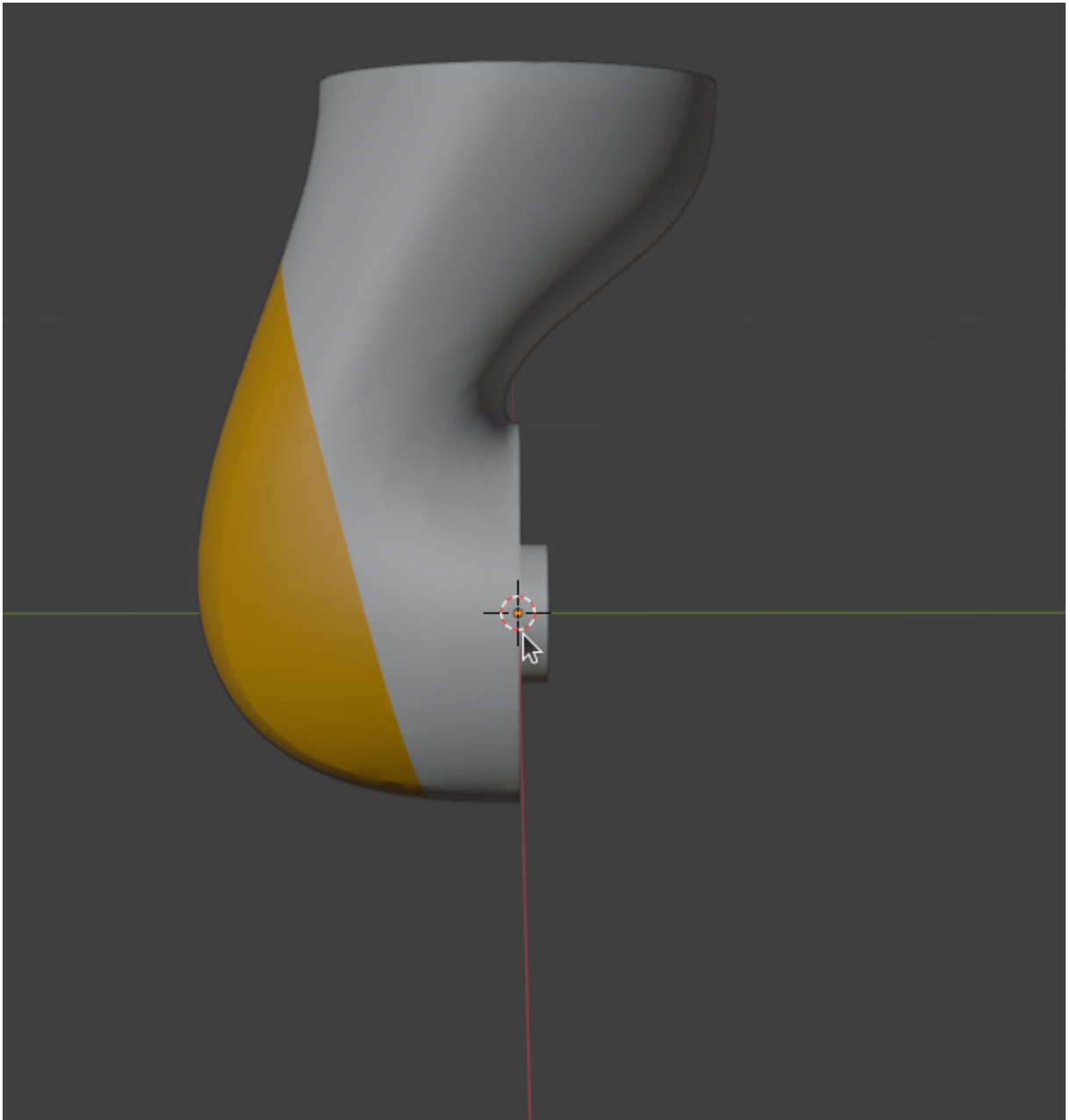
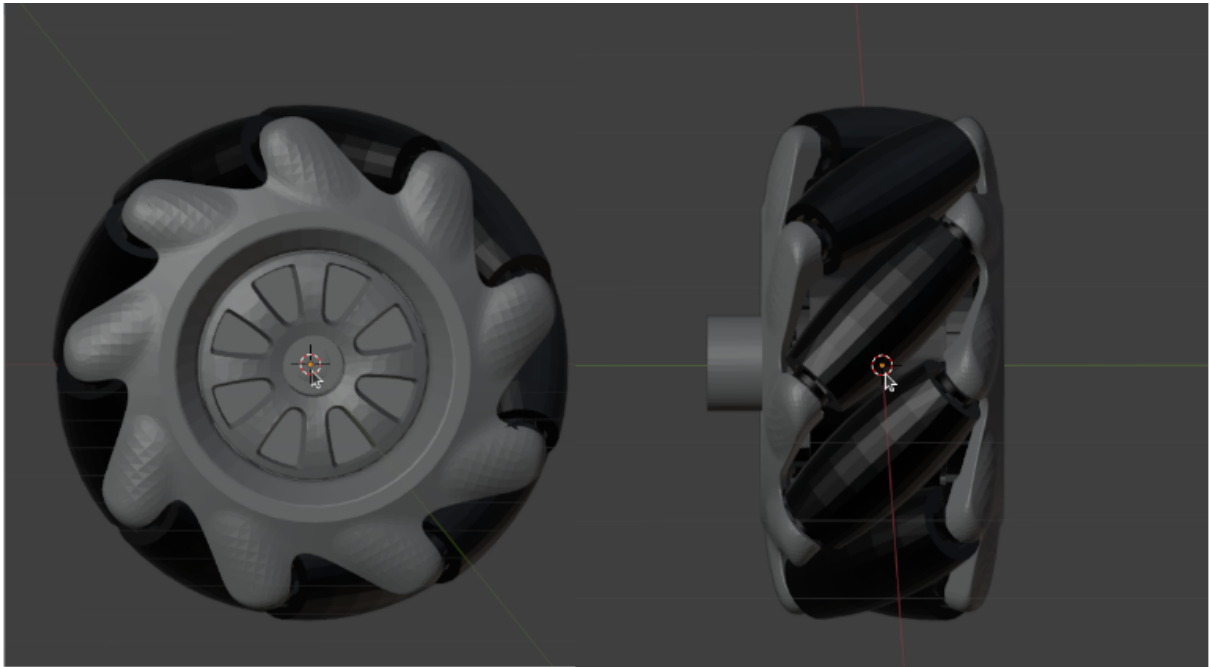


Figure 1.7: Wheel origin point



Procedure to Move the Mesh While Keeping the Origin Fixed

To move the geometry without altering the global position of the orange dot, follow the steps below:

1. **Enter Edit Mode:** Select the specific part and press **TAB** to switch from *Object Mode* to *Edit Mode*.
2. **Select the Entire Mesh:** Press **A** to select all vertices, edges, and faces. The part will be fully highlighted.
3. **Move the Mesh Relative to the Origin:** Press the **G** key followed by the desired axis (**X**, **Y**, or **Z**) to move the part's geometry. Move it until the fixed orange dot is exactly at the desired joint axis or center. Press **TAB** to exit Edit Mode when finished.

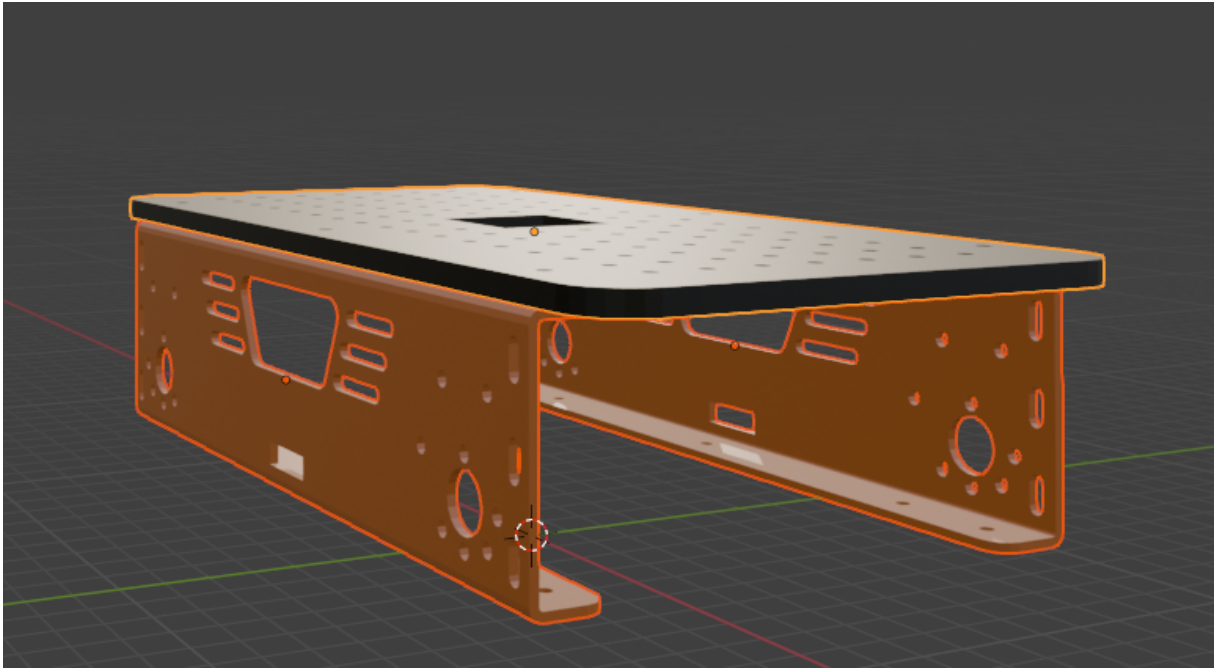
1.1.4 Step 4: Joining Static Parts (Join)

After configuring the individual parts, we can group elements that remain fixed relative to each other (meaning they have no relative motion). In **Figure 1.8**, for example, we have two orange sides and a rectangular plate that will always move together as part of the robot; they have no joints between them, so they can become a single object.

Important Attention: When joining parts, the origin point (orange dot) of the newly unified object will be located at the position of the **last individual object you selected** (the active object, which has a light yellow outline, as shown in **Figure 1.8**).

Make sure that the last selected object is the one holding the correct origin for the URDF, or readjust the origin after joining.

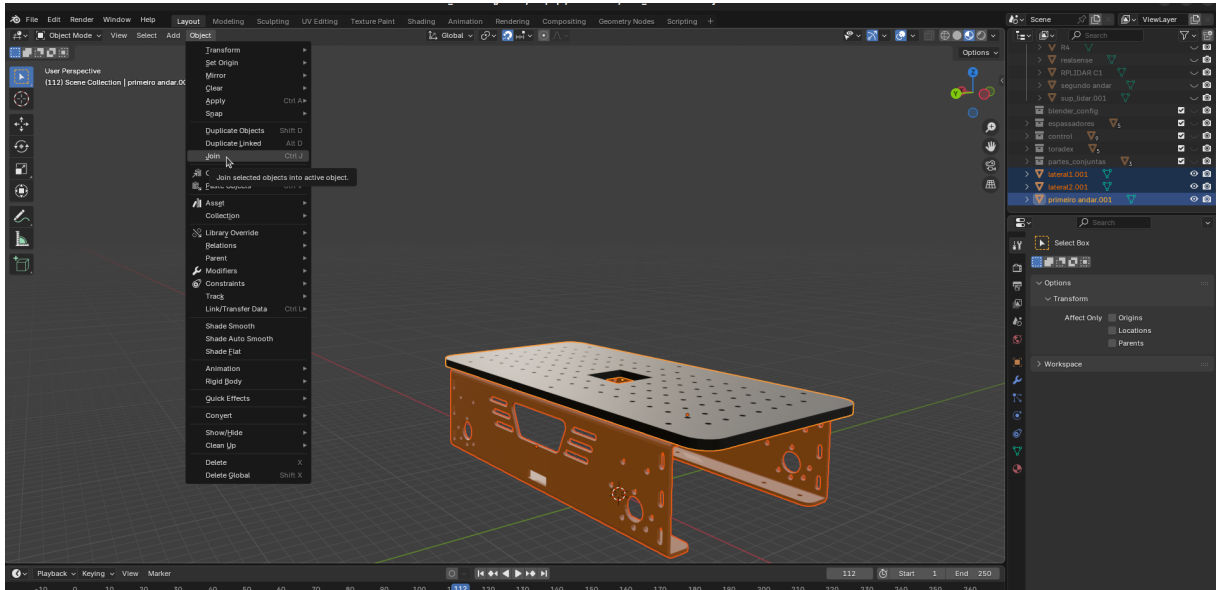
Figure 1.8: Selecting parts (rectangular plate selected last)



To perform the join safely (preserving originals if necessary):

1. Select all parts to be joined.
2. Create a backup copy by pressing **Ctrl + C** and **Ctrl + V**.
3. With the copies selected, go to the top menu **Object** → **Join** (or use the shortcut **Ctrl + J**), as shown in **Figure 1.9**.

Figure 1.9: Unified parts



1.1.5 Step 5: Exporting to COLLADA (.dae)

With individual parts and static assemblies properly adjusted, the final step is exporting to the .dae format, widely used in URDF for supporting materials and complex meshes efficiently.

To ensure the URDF positions the part correctly, we export the mesh aligned to Blender's global origin (0, 0, 0):

1. Select the desired part and temporarily change its location (*Location*) in the *Sidebar* to 0, 0, 0.
2. Go to **File** → **Export** → **Collada (.dae)**, shown in **Figure 1.10**.
3. Choose the destination folder inside your ROS package.
4. In the export panel, enable the **Selection Only** option to export strictly the active part, as shown in **Figure 1.11**.
5. After saving, return the part to its original position in the Blender scene using the shortcut **Ctrl + Z**.
6. Repeat the same process for all other parts of the robot.

Figure 1.10: Exporting the .dae file

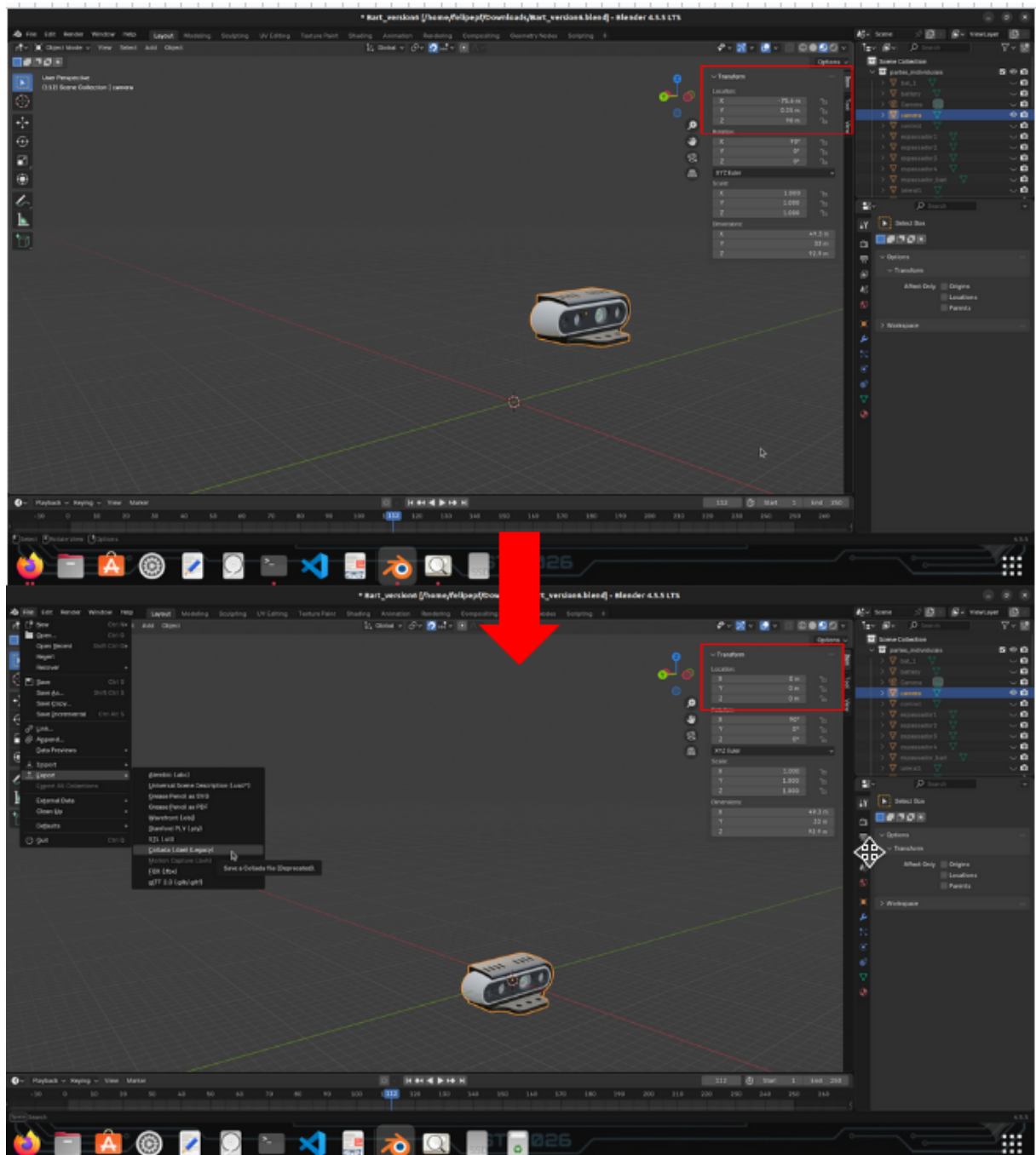
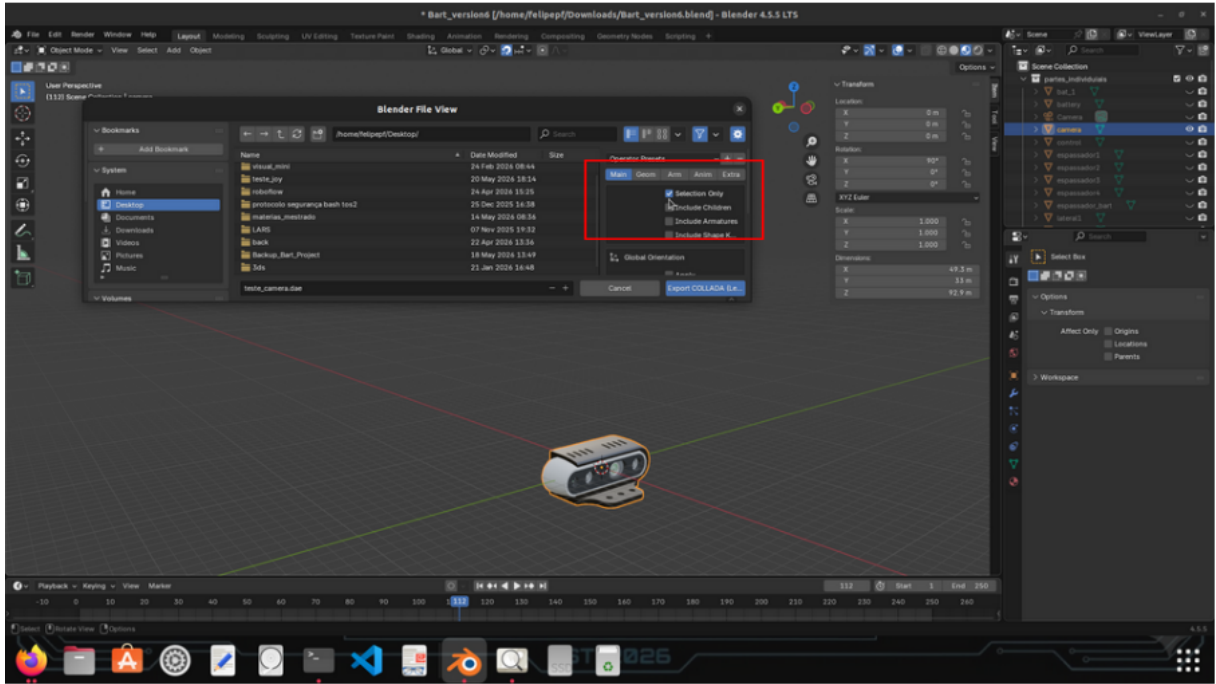


Figure 1.11: Export screen



With all parts adjusted, scales corrected, origins well-positioned, and structurally exported, your model is ready to be mapped in the URDF file.

1.1.6 Importing, Optimizing, and Customizing STL Files

Often, structural robot meshes are designed in CAD software and exported in the STL (*Stereolithography*) format. While popular for manufacturing and 3D printing, STL has some critical limitations for ROS simulations (such as lack of native color/material support and excessively dense meshes).

In this section, we cover the correct procedure for importing, optimizing topology, and visually customizing your STL files inside Blender.

Step 1: Importing the STL File

To bring your CAD model into the preparation environment:

1. Go to the top menu and click **File** → **Import** → **Stl (.stl)** (as shown in **Figure 1.1**).
2. Navigate to the folder where the file is saved, select it, and click *Import STL*.

Scale Note: As soon as the file is imported, remember to apply the dimension validation and scale adjustment concepts discussed in the previous section, ensuring the object is neither massive nor invisible in the scene.

Step 2: Aesthetic Customization (Materials and Colors)

Since STL files do not carry color information, the imported model will appear entirely gray. To give your robot a professional visual identity:

1. Select the desired part and go to the **Material Properties** tab (the checkered sphere icon on the right side menu).
2. Click **New** to create a new material, as shown in **Figure 1.12**.
3. In the **Base Color** field, select the desired color for your *link*, as shown in **Figure 1.13**.
4. If you wish to assign different colors to specific parts of the object, simply enter edit mode (*Edit Mode*), select the desired face or faces, and access the **Material Properties** tab. Then, click **New** (or the + icon), choose the desired color in the **Base Color** field, and finally, click **Assign**, as shown in **Figure 1.14**.

Figure 1.12: Creating a new material

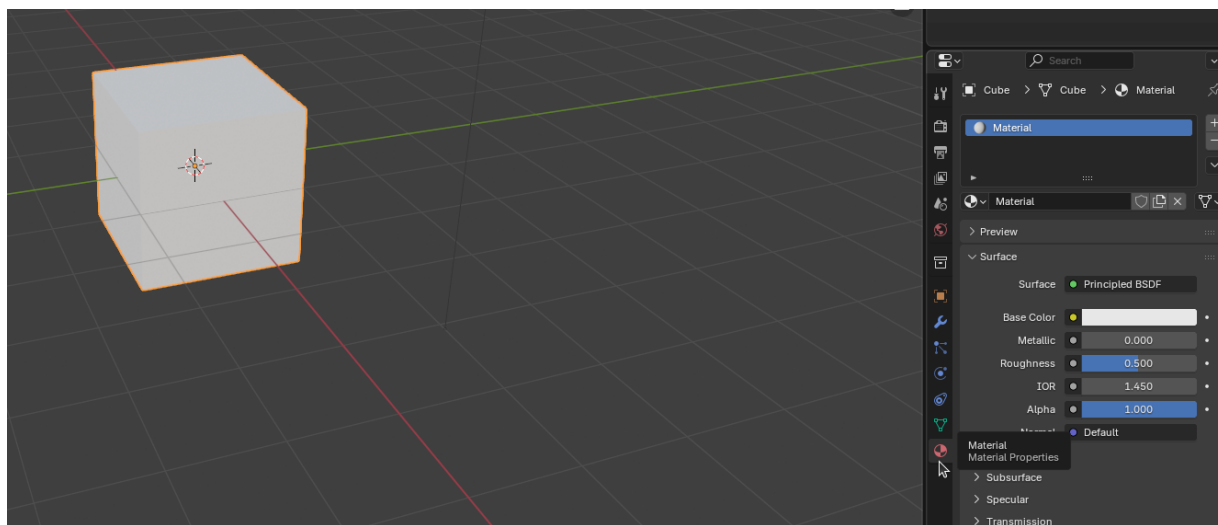


Figure 1.13: Modifying the color with a new material

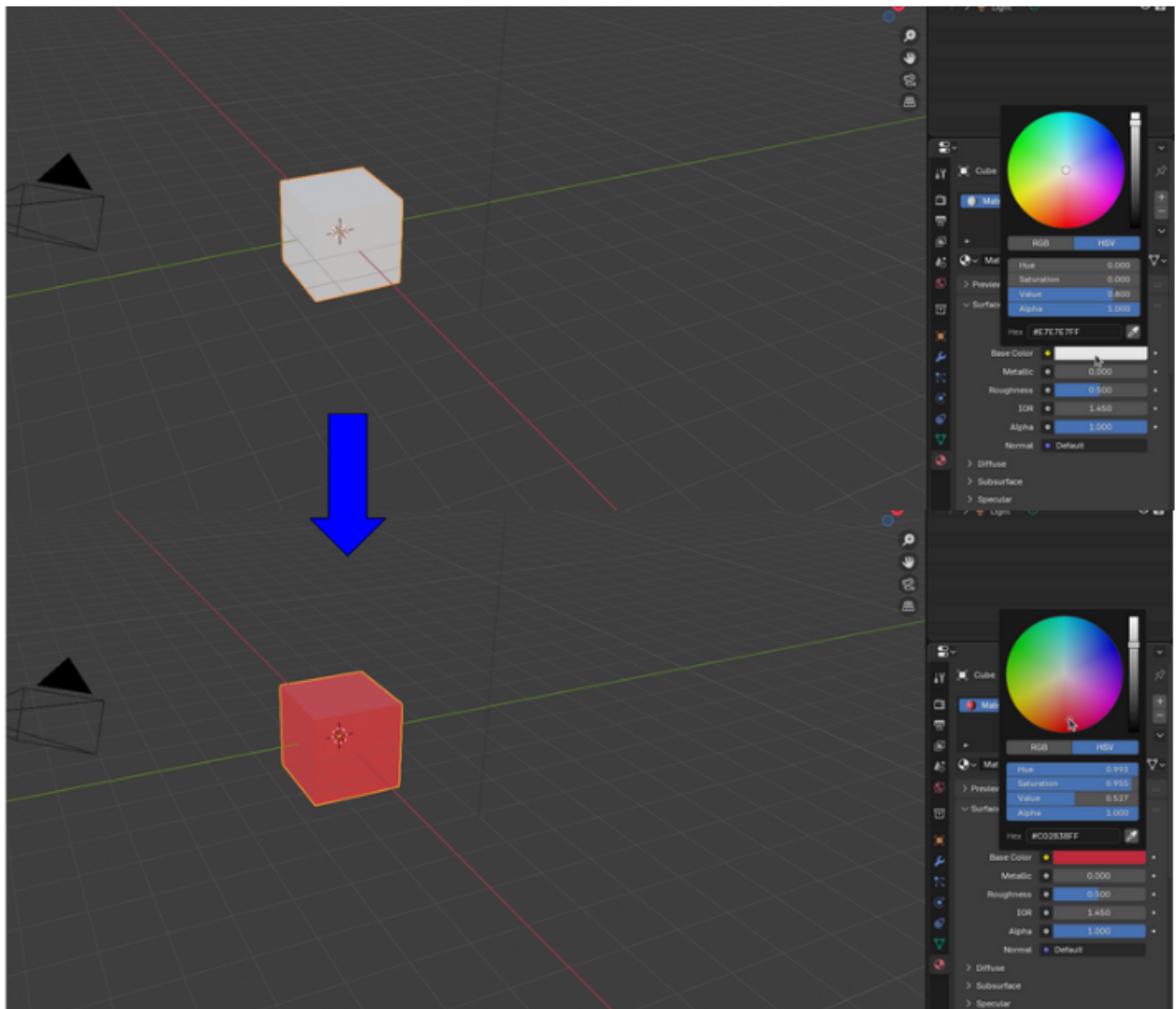
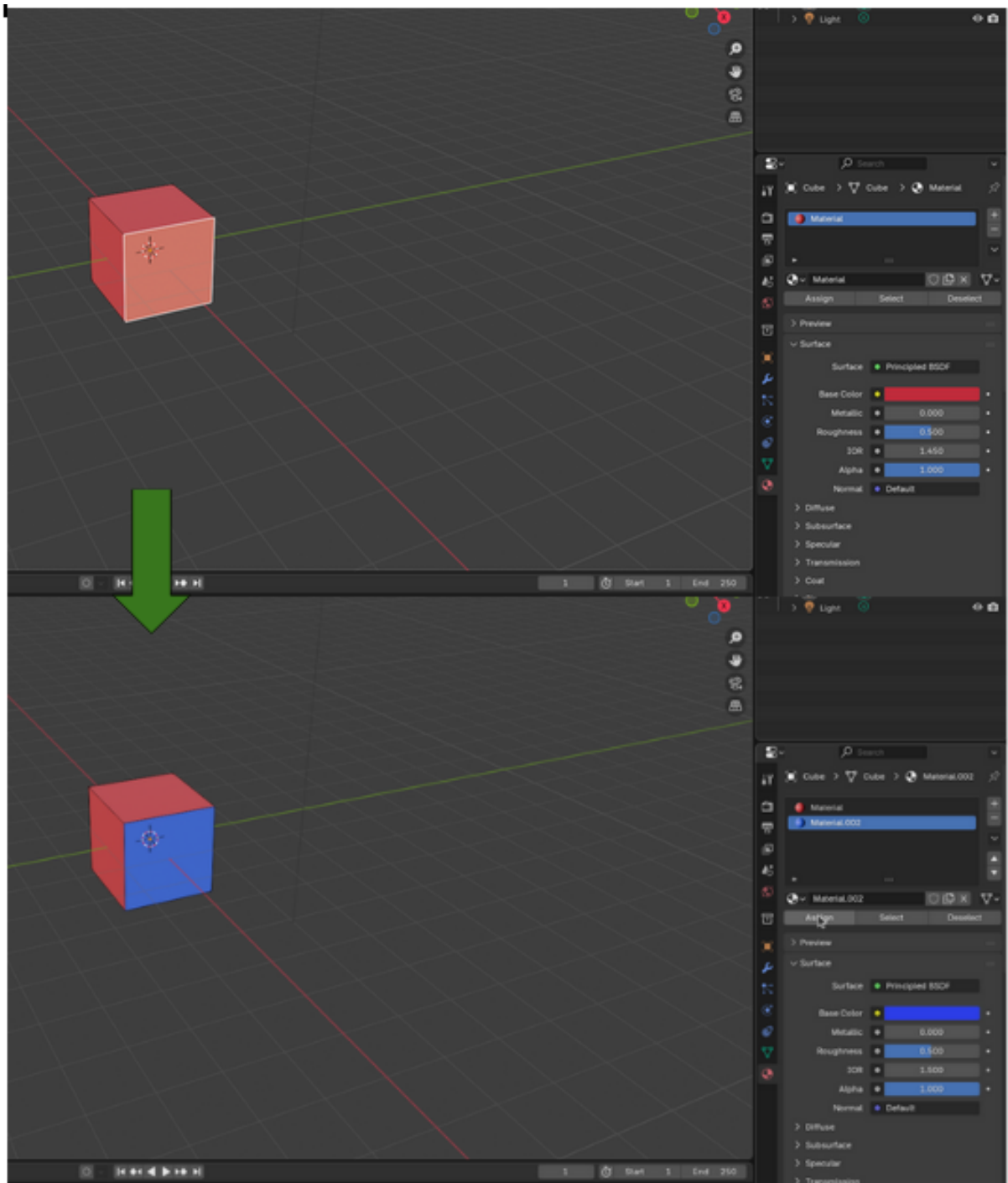


Figure 1.14: Implementing different colors



2 FOLDER CREATION, URDF ASSEMBLY, AND ENVIRONMENT CONFIGURATION

After preparing, optimizing, and exporting all geometric meshes in .dae format inside Blender, the next essential step is to integrate them into the ROS 2 development structure.

In this chapter, we will learn how to create the correct directory tree for the robot, how to structure the initial skeleton of the URDF file, and how to configure the development environment so that the system recognizes your model.

2.0.1 Project Folder Structure

In ROS 2, robot models must be organized within a dedicated package, for example, *nome_do_robo_proj*. To do this, navigate to the source (*source*) directory of your *workspace* and use the ROS 2 command-line tool to generate the structured package with Python support and an Apache-2.0 license.

The step-by-step procedure in the terminal consists of the following commands:

1. Navigate to the workspace:

```
cd ~/workspace_ws/src
```

2. Create the ROS 2 package: Run the command below, replacing *nome_do_robo_project* with your project's name. The `-build-type ament_python` parameter ensures compatibility with launch files and Python automation, while `-license` defines the terms of use:

```
ros2 pkg create nome_do_robo_project \
  --build-type ament_python \
  --license Apache-2.0
```

3. Create the necessary directory tree: Enter the newly created package folder and use the `mkdir` command to generate all structural subfolders required for organizing the robot, including visual mesh subdirectories:

```
cd nome_do_robo_project
mkdir -p config launch rviz meshes/visual world urdf script
```

With this, the base structure of your package is ready and standardized. The `meshes/visual` folder should host the `.dae` files exported from Blender, the `urdf` folder will house the structural description of the robot, and the `launch` and `rviz` directories will contain the launch scripts and visual configuration files of the ecosystem, respectively.

2.0.2 Configuration of Manifesto and Build Files

After creating the physical directory structure, it is critical to configure the build files of ROS 2. In packages using the *ament_python* build system, software dependency management and resource mapping to the system's installation directory depend directly on the correct editing of *package.xml* and *setup.py*, in addition to including the manifest file for the Gazebo simulator, *model.config*.

Step 1: Dependency Management in *package.xml*

The *package.xml* file formally defines the package's properties and lists all execution dependencies (*exec_depend*) required for teleoperation nodes, state publishers, and control systems to function in an integrated manner. Open the file located at the root of your package and configure it by adding the robotics and simulation dependencies, as shown in the following example:

```
<?xml version="1.0"?>
<package format="3">
  <name>your_pkg_name</name>
  <version>0.0.0</version>
  <description>A brief description</description>

  <maintainer email="your_email">your_name</maintainer>
  <license>Apache-2.0</license>

  <!-- Build system -->
  <buildtool_depend>ament_python</buildtool_depend>

  <!-- Runtime dependencies -->
  <exec_depend>rclpy</exec_depend>
  <exec_depend>geometry_msgs</exec_depend>
  <exec_depend>nav_msgs</exec_depend>
  <exec_depend>tf2</exec_depend>
  <exec_depend>tf2_ros</exec_depend>

  <!-- Robot / visualization -->
```

```

<exec_depend>robot_state_publisher</exec_depend>
<exec_depend>joint_state_publisher</exec_depend>
<exec_depend>xacro</exec_depend>
<exec_depend>rviz2</exec_depend>
<exec_depend>ros2_control</exec_depend>
<exec_depend>ros2_controllers</exec_depend>
<exec_depend>controller_manager</exec_depend>
<exec_depend>diff_drive_controller</exec_depend>
<exec_depend>joint_state_broadcaster</exec_depend>
<exec_depend>teleop_twist_keyboard</exec_depend>
<exec_depend>gazebo_ros2_control</exec_depend>

<!-- Testing -->
<test_depend>ament_copyright</test_depend>
<test_depend>ament_flake8</test_depend>
<test_depend>ament_pep257</test_depend>
<test_depend>python3-pytest</test_depend>

<export>
  <build_type>ament_python</build_type>
</export>
</package>

```

Step 2: Resource Mapping in *setup.py*

Unlike CMake-based packages, a Python package requires the *setup.py* script to explicitly point to where mesh files, RViz configurations, maps, and simulator worlds are located. This ensures that *colcon build* can export and index these resources in the shared ROS 2 directory (*share/*).

Open the *setup.py* file and restructure the *data_files* field, using the *glob* library to dynamically gather the contents of each project subfolder:

```

from setuptools import setup, find_packages
import os
from glob import glob

package_name = 'your_pkg_name'

setup(
    name=package_name,

```

```

version='0.0.0',

# Descriptive package -> does not need Python modules for basic ins
packages=find_packages(),

data_files=[
    # Register package in ament index
    ('share/ament_index/resource_index/packages',
     ['resource/' + package_name]),

    ('share/' + package_name, ['model.config']),

    # package.xml
    ('share/' + package_name,
     ['package.xml']),

    # Launch files
    (os.path.join('share', package_name, 'launch'),
     glob('launch/*.py')),

    # URDF / XACRO
    (os.path.join('share', package_name, 'urdf'),
     glob('urdf/*')),

    (os.path.join('share', package_name, 'config'),
     glob('config/*')),

    # Meshes VISUAL
    (os.path.join('share', package_name, 'meshes', 'visual'),
     glob('meshes/visual/*.*')),

    # Worlds
    (os.path.join('share', package_name, 'worlds'),
     glob('worlds/*.world')),

    # Rviz
    (os.path.join('share', package_name, 'rviz'),

```

```

        glob('rviz/*.rviz')),
    ],

    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='your_name',
    maintainer_email='your_email',
    description='A brief description',
    license='Apache License 2.0',
    entry_points={
        'console_scripts': [],
    },
)

```

Step 3: Simulator Integration via *model.config*

For simulators like Gazebo Classic to automatically identify the robot model within their local model library (GAZEBO_MODEL_PATH), the package directory must contain a simplified manifest file named exactly *model.config*.

Create this file in the root of your package to declare the entry point of your processed URDF file:

```

<?xml version="1.0"?>
<model>
  <name>your_robot_name</name>
  <version>1.0</version>
  <sdf version="1.7">urdf/processed/bart_version1.urdf</sdf>
  <author>
    <name>your_name</name>
    <email>your_email</email>
  </author>
  <description>
    A brief description
  </description>
</model>

```

2.0.3 Understanding the Anatomy of URDF: What are Links and Joints?

For those beginning in robotics, a URDF file can seem like a complex tangle of XML code. However, its logic is purely mechanical. A URDF describes a robot as a kinematic tree composed of two fundamental entities:

- **Links (Bodies/Links):** Represent the rigid, physical parts of the robot (e.g., the chassis, an aluminum arm, a wheel).
- **Joints (Articulations/Joints):** Represent the connection points that join one Link to another, dictating how they can move relative to each other (e.g., a bearing, a hinge, a motor).

Each individual **Link** is internally divided into three essential blocks that tell the computer what the part looks like, where it collides, and how much it weighs: **Visual**, **Collision**, and **Inertial**. Below, we detail the structure of each.

1. The Visual Block (<visual>) — The Robot’s Appearance

The <visual> block serves exclusively for rendering and graphical visualization (such as in RViz2 and simulator cameras). It defines **what we see** on the screen.

- **What to place here:** This is where we insert the complex, detailed, and textured meshes exported from Blender (.dae).
- **Technical caution:** This block has no physical properties. If you place a huge and heavy visual mesh, the robot will look robust on screen, but to the physics simulator, it remains invisible and intangible.

2. The Collision Block (<collision>) — The Physical Body

The <collision> block defines the geometry (ODE, Bullet, PhysX) used to calculate impacts, ground contact, and environmental obstacles. It determines **where the robot collides**.

- **What to place here:** Instead of using Blender’s complex mesh (which would require millions of mathematical calculations to process collisions), we use **simple primitive geometries** (such as boxes <box>, cylinders <cylinder>, or spheres <sphere>) that approximate the part’s outer boundaries.
- **Why do this:** If the robot’s chassis collides with a wall, the simulator calculates the impact based on the invisible box of the <collision> block, saving computer processing and preventing the robot from phasing through objects or falling through the ground.

3. The Inertial Block (<inertial>) — Physics and Mass

The <inertial> block dictates gravity and acceleration forces acting on the robot. It determines **how difficult it is to move or stop the part**. It consists of three mathematical parameters:

- **Mass (<mass>):** The actual mass of the component in kilograms (kg). If you state the chassis weighs 1.5 kg, the motor forces must be proportionally sized to move it from rest.

- **Origin (<origin>):** Indicates the X, Y, Z coordinate of the **Center of Mass** (center of gravity) of the part relative to the Link's origin. If the center of mass is shifted to the left side, the robot will tend to tip left under gravity.
- **Inertia Matrix (<inertia>):** Represented by the rotational moments of inertia (I_{xx}, I_{yy}, I_{zz}). While mass measures resistance to linear motion, inertia measures **resistance to rotation** around its own local axes.

4. The Joint Structure (<joint>) — The Motion Constraint

If Links are the bones, the <joint> is the joint. A joint creates a hierarchical dependency relationship between two parts: the Parent Link (**parent**) and the Child Link (**child**).

The basic structure and meaning of its tags are explained below:

- **parent** (Parent Link): Serves as the static reference or base. If the Parent Link moves, the Child Link moves rigidly along with it.
- **child** (Child Link): The moving part (or attached piece) that receives the motion generated by the joint.
- **origin** (Joint Origin): Defines the exact position (coordinates X, Y, Z in meters) and rotation (*Roll, Pitch, Yaw* angles in radians) where the articulation point (that orange dot configured in Blender) will be positioned relative to the Parent Link's origin.
- **axis** (Axis of Motion): Specifies the unit vector (X, Y, Z) around which (or along which) the motion occurs. For example, if the axis is `xyz="0 1 0"`, it means the wheel or joint rotates strictly around the local Y axis of the joint.
- **limit** (Physical Limits): Restricts joint operation by defining the maximum torque supported by the motor (`effort`) and the maximum angular or linear velocity allowed (`velocity`).

The URDF features 6 primitive joint types:

- **fixed** (Fixed):
 - **What it does:** Locks all relative motion between the two parts. They behave as if rigidly welded or bolted together.
 - **Common use:** Connecting sensors (such as cameras or LiDARs) to the main chassis, or joining static sub-structures of the robot body.
- **continuous** (Continuous):
 - **What it does:** Allows pure continuous rotation around a single specified axis ****without limits**** (can rotate 360 degrees infinitely). It has no limit stops.

- **Common use:** Modeling wheels on mobile robots (differential, omnidirectional, etc.) or transmission pulleys.
- **revolute** (Revolute):
 - **What it does:** Allows rotation around a single axis but has ****strict angular limits**** (minimum and maximum angles expressed in radians in the code).
 - **Common use:** Joints of industrial robot arms (shoulder, elbow, wrist), gripper claws, and pan-tilt camera systems.
- **prismatic** (Prismatic):
 - **What it does:** Instead of rotating, it allows ****linear translation**** (sliding in a straight line) along a specific axis. It also requires the configuration of physical limits (minimum and maximum in meters).
 - **Common use:** Linear actuators, pneumatic pistons, cargo elevators on warehouse robots, and 3D printer axes.
- **planar** (Planar):
 - **What it does:** Allows a part to slide and rotate freely on a two-dimensional plane. It gains three degrees of freedom within that plane: translation in X , translation in Y , and rotation (yaw) around the Z axis.
 - **Common use:** Simplified tracking of omnidirectional bases moving directly on flat floors without modeling individual wheel friction.
- **floating** (Floating):
 - **What it does:** Allows completely free motion in 3D space, meaning it unlocks all 6 degrees of freedom (3 linear translations and 3 angular rotations). No geometric constraints exist between parent and child.
 - **Common use:** Used to model aerial robots (like drones and quadcopters) or the floating base of humanoid and quadruped robots before making foot contact with the ground.

By combining these 6 joint types and adding appropriate plugins to read sensors and actuate motors, you can simulate and program everything from simple remote-control rovers to highly complex industrial and humanoid robots in ROS 2!

1. What are Simulation Plugins? — The Software Bridge

Imagine you designed your robot perfectly in the URDF. It has weight, wheels, and a camera. However, if you simply spawn it inside the simulator (like Gazebo), it will act like a statue: wheels won't rotate when speed commands are sent, and the camera won't output any image to your Computer Vision code.

This is where **Plugins** come in. A plugin is a piece of software (typically a compiled C++ library with a `.so` extension) that acts as a **virtual electronic bridge**. It connects the simulator's physics engine to ROS 2 nodes and topics.

Plugins are broadly divided into two functional categories:

1. **Actuation and Motion Plugins (Virtual Drivers):** They simulate the control electronics and motors of the robot. In practice, this plugin “listens” to ROS 2 velocity command topics (like `/cmd_vel`). When a user sends a motion command via keyboard or navigation algorithm, the plugin translates it into physical forces and speeds applied directly to joint models inside the simulation. Additionally, it calculates **Odometry** (the estimation of where the robot is in space) and publishes this information back to ROS 2.
2. **Sensor Plugins (Virtual Hardware):** They simulate the behavior of real physical sensors by capturing data generated within the virtual environment.
 - *RGB-D Camera Example:* The plugin reads the pixel array from the virtual simulator environment, calculates 3D depth for each object, and packages this data into raw images and *Point Clouds*. It then publishes these data streams to standard ROS 2 topics, enabling computer vision or SLAM algorithms to run exactly as they would with physical hardware connected.

With this clear conceptual foundation, reading any URDF code becomes intuitive: you are mapping what the robot displays visually, its simplified physical collision shape, its mass distribution, and the mechanical joint rules linking its parts.

2.1 The Golden Rule of URDF Assembly (Relative Coordinate Reference)

Think of URDF assembly exactly like a classic Classical Mechanics problem: **Relative Position**.

When calculating the displacement of an object, we use:

$$\Delta X = X_{\text{final}} - X_{\text{initial}}$$

In URDF, the logic is identical. When you create a joint, you define where the child link (*child*) is located relative to its parent link (*parent*). The starting reference point (origin) is no longer the “center of the floor” (global origin) but rather the origin of the parent link.

The Golden Rule Formula:

$$\text{Joint Coordinate} = \text{Child Position in the World} - (\text{Parent Position in the World})$$

Practical Example: Analyzing the Error and the Correction

Imagine we have the **Lower Part** (Chassis) and the **Upper Part** of the robot.

In the 3D modeling software (where the global origin (0, 0, 0) is the floor):

- The origin of the **Lower Part** is at: $Z = 0.042$ m (in the world)
- The origin of the **Upper Part** is at: $Z = 0.131$ m (in the world)

The Wrong Way (Direct Copy and Paste)

If you only input the global coordinate of the upper part into the *joint* origin:

```
<joint name="parte_superior_joint" type="fixed">
  <parent link="parte_inferior_link"/>
  <child link="parte_superior_link"/>
  <origin xyz="0.001 0.003 0.131" rpy="0.0 0.0 0.0"/>
</joint>
```

What happens: ROS will interpret that the upper part is 0.131 m above the lower part (which in turn is already suspended in the air). The upper part will appear floating far away from its correct assembly location!

The Correct Way (Applying the Golden Rule)

Since the *parent* is `parte_inferior_link`, we need to calculate the relative distance between the two parts:

$$\Delta Z = Z_{\text{upper}} - (Z_{\text{lower}})$$

$$\Delta Z = 0.131 \text{ m} - 0.042 \text{ m} = 0.089 \text{ m}$$

Now, applying the correct relative offset in the URDF:

```
<joint name="parte_superior_joint" type="fixed">
  <parent link="parte_inferior_link"/>
  <child link="parte_superior_link"/>
  <origin xyz="0.002 0.007 0.089" rpy="0.0 0.0 0.0"/>
</joint>
```

Note: 0.007, because: 0.004 - (-0.003) and 0.002, because: 0.001 - (-0.001)

Summary

- **The Parent Link is your new “Zero” (0, 0, 0):** When configuring a *joint*, forget Blender’s floor. Act as if the universe begins at the center of the parent link.
- **Perform Subtraction:** Always subtract the global coordinates of the parent part from the global coordinates of the child part (Child – Parent). The result of this calculation is what goes into the `<origin xyz="..." />` field of the joint.

3 PRACTICAL CODE EXAMPLE

Below is the minimal declaration essential to initialize a robot. This practical example demonstrates how to connect a chassis link to a wheel using a continuous joint, serving as a baseline template to understand URDF syntax:

```
<?xml version="1.0"?>
<robot name="robo_minimalista" xmlns:xacro="http://ros.org/wiki/xacro">

  <!-- 1. VIRTUAL GROUND PROJECTION LINK -->
  <link name="base_footprint"/>

  <!-- 2. FIXED CONNECTION JOINT OF THE BASE -->
  <joint name="base_joint" type="fixed">
    <parent link="base_footprint"/>
    <child link="chassi_link"/>
    <origin xyz="0.0 0.0 0.05" rpy="0.0 0.0 0.0"/>
  </joint>

  <!-- 3. MAIN CHASSIS LINK -->
  <link name="chassi_link">
    <inertial>
      <mass value="1.5"/>
      <origin xyz="0.0 0.0 0.0"/>
      <inertia ixx="0.005" ixy="0.0" ixz="0.0" iyy="0.005" iyz="0.0" iz
    </inertial>
    <collision>
      <geometry>
        <box size="0.3 0.2 0.1"/>
      </geometry>
    </collision>
    <visual>
      <geometry>
        <box size="0.3 0.2 0.1"/>
      </geometry>
    </visual>
  </link>

  <!-- 4. CONTINUOUS JOINT FOR THE WHEEL -->
```

```

<joint name="roda_esquerda_joint" type="continuous">
  <parent link="chassi_link"/>
  <child link="roda_esquerda_link"/>
  <origin xyz="0.0 0.12 -0.02" rpy="0.0 0.0 0.0"/>
  <axis xyz="0.0 1.0 0.0"/>
</joint>

<!-- 5. WHEEL LINK -->
<link name="roda_esquerda_link">
  <inertial>
    <mass value="0.2"/>
    <origin xyz="0.0 0.0 0.0"/>
    <inertia ixx="0.0001" ixy="0.0" ixz="0.0" iyy="0.0001" iyz="0.0"
  </inertial>
  <collision>
    <origin xyz="0.0 0.0 0.0" rpy="1.5708 0.0 0.0"/>
    <geometry>
      <cylinder radius="0.03" length="0.02"/>
    </geometry>
  </collision>
  <visual>
    <origin xyz="0.0 0.0 0.0" rpy="1.5708 0.0 0.0"/>
    <geometry>
      <cylinder radius="0.03" length="0.02"/>
    </geometry>
  </visual>
</link>

</robot>

```

3.0.1 Line-by-Line Analysis of the Example Code

To ensure complete understanding of the code above, let's break down the purpose of each declared block:

- **XML Header and <robot> tag:** The first line indicates that the file complies with the XML standard. The <robot> tag bounds our robot's scope and defines its official name for ROS 2 (robo_minimalista). The `xmlns:xacro` parameter extends the file to accept macros in the future.

- **The Ground Origin (`base_footprint`):** The robot starts with an empty link on the ground. The `base_joint` joint is fixed (does not move) and simply states that the physical body of the robot (`chassi_link`) floats 0.05 meters (5 centimeters) above the ground (Z-axis).
- **The Chassis Block (`chassi_link`):** Modeled as a box (`box`) with dimensions $30 \times 20 \times 10$ centimeters (`size="0.3 0.2 0.1"`). Notice that the same size is mirrored in both `<visual>` and `<collision>`, ensuring the collision box occupies the exact same space we see on screen. It has a defined mass of 1.5 kg.
- **The Wheel Joint (`roda_esquerda_joint`):** Connects the chassis to the wheel. Since wheels spin freely, the chosen type is `continuous`.
 - The `origin` tag offsets the wheel 12 centimeters to the left ($Y = 0.12$) and 2 centimeters down ($Z = -0.02$) relative to the chassis center.
 - The `axis xyz="0 1 0"` tag is crucial: it tells the physics engine that this wheel can only rotate around its local Y axis (longitudinal axis of the bearing).
- **The Wheel Block (`roda_esquerda_link`):** Geometrically modeled as a cylinder (`cylinder`) with a radius of 3 cm and width of 2 cm.
 - *Important orientation detail:* By default, URDF draws cylinders upright (aligned with the Z-axis). Since wheels must be laid flat to roll on the ground, a rotation of 1.5708 radians (90°) was applied to the `roll` (`Roll`) parameter inside both visual and collision blocks to align it properly along the local X-axis.

4 ENVIRONMENT AUTOMATION WITH LAUNCH FILES

In ROS 2, running your robot requires more than having structural models ready. You need to execute multiple software nodes in parallel: a node to read and publish the URDF, another to manage joint states, and finally, visual simulation interfaces. Running each of these manually in separate terminals is impractical.

To solve this, we use Python-based **Launch Files**. They act as orchestration scripts that configure parameters, resolve package paths, and initialize the entire robot ecosystem with a single terminal command.

4.0.1 The Role of Core Initialization Nodes

Before analyzing the launch code, it is crucial to understand the two native ROS 2 nodes that support robot visualization:

- **robot_state_publisher**: This node is the heart of kinematics in ROS 2. It receives the complete URDF file (or processed XACRO) as a text parameter. Its function is to continuously compute the 3D spatial transformations (translation and rotation coordinates) between all robot *links*. It publishes this data to the global transform system of ROS 2, known as **TF** (*Transform Library*). Without this node, RViz2 would not know where the arm or wheels are positioned relative to the chassis.
 - **joint_state_publisher_gui**: When testing a robot only in RViz2 (without an active physical simulator like Gazebo), moving joints of type `revolute` or `continuous` lack virtual motors sending angle data. This node automatically creates a simple GUI with slider bars. Moving these sliders publishes simulated angle data to the `/joint_states` topic, letting you test if wheels and links rotate on the correct axes before running physical simulations.
-

4.0.2 Practical Example: Launch Script (`display.launch.py`)

Create a file named `display.launch.py` inside the `launch/` folder of your package. This script will process the structural XACRO file of the robot, initialize the state publishers, and launch RViz2:

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
```

```

from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import Node
import launch

def generate_launch_description():

    # 1. Package name and local path definitions
    package_name = 'your_package_name'
    pkg_share = get_package_share_directory(package_name)

    # Path to the main URDF.XACRO robot file
    xacro_file = os.path.join(pkg_share, 'urdf', 'file.urdf.xacro')

    # Process the XACRO file via system command
    # (Generates clean XML text with macros resolved)
    robot_description_raw = launch.substitutions.Command(['xacro ', xacro_file])

    # 2. Configure and include native Gazebo Launch
    # Find the default gazebo_ros package to load the simulator
    pkg_gazebo_ros = get_package_share_directory('gazebo_ros')

    # Includes the default Gazebo launch script (opens an empty world)
    launch_gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(pkg_gazebo_ros, 'launch', 'gazebo.launch.py')
        )
    )

    # 3. Declare ROS 2 Nodes for execution

    # Robot State Publisher Node (Required for Gazebo to parse kinematic data)
    node_robot_state_publisher = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        output='screen',
        parameters=[{
            'robot_description': robot_description_raw,
            'use_sim_time': True # Syncs node clock with simulator clock
        }]
    )

```

```

    ]]
)

# Spawn Node: Injects the robot entity into Gazebo
# It reads the published 'robot_description' and spawns it in the world
node_spawn_entity = Node(
    package='gazebo_ros',
    executable='spawn_entity.py',
    arguments=[
        '-topic', 'robot_description', # Topic to read the URDF from
        '-entity', 'robot_name',      # Name of the robot in the world
        '-x', '0.0',                  # Initial spawn coordinate x
        '-y', '0.0',                  # Initial spawn coordinate y
        '-z', '0.05',                 # Initial spawn coordinate z
    ],
    output='screen'
)

# 4. Return Launch Description containing Gazebo and Nodes
return LaunchDescription([
    launch_gazebo,
    node_robot_state_publisher,
    node_spawn_entity
])

```

4.0.3 Analysis of the Spawn Process

- **The importance of `use_sim_time`:** When running robots in physics simulators, the real computer clock (CPU time) and the physics simulation clock can drift slightly. Setting `'use_sim_time': True` forces the `robot_state_publisher` node to synchronize joint math calculations with Gazebo's internal clock. If omitted, TF frames will drift, breaking SLAM and navigation pipelines.
- **The `spawn_entity.py` script:** This community utility in the `gazebo_ros` package removes the need to manually convert XACRO files to Gazebo-native SDF files. The `-topic robot_description` argument retrieves the XML string already processed by the `robot_state_publisher` from RAM, injecting the robot smoothly at runtime.
- **Positive Z Coordinate (`-z 0.05`):** It is best practice in simulation engineering to spawn the robot slightly above the ground (5 centimeters in this example). This prevents wheels from colliding violently with the ground plane at the exact millisecond physics are ini-

tialized, which could otherwise launch the robot upward due to initial numerical contact instability.

5 ADAPTATION FOR ISAAC SIM

5.0.1 How to Convert Xacro Files to Static URDFs

With the final URDF file validated via launch files in ROS 2, the next step is preparing it for the NVIDIA Isaac Sim environment. Since this simulator requires a static, fully resolved URDF file for import, we must process the Xacro macros beforehand.

To do this, navigate to the directory where your `your_model.urdf.xacro` file is located, open a terminal, and run:

```
xacro your_model.urdf.xacro > your_model.urdf
```

This command processes all Xacro properties and macros, outputting a clean, completed URDF file ready to be imported directly into Isaac Sim using the *URDF Importer* tool.